Progress Report: Longest Loop Under a Distance Constraint

Grace Yanzito, Lucy Dobson, Hannah Bregel

Github Link: `https://github.com/GraceYanzito/Analysis-of-Algorithms-Project`

1 Question

What is the longest loop given a distance constraint?

2 Preamble

We are trying to find the longest path given a distance constraint in a National Park. The only modification we made to our question is substituting distance for time. For the prototype there were too many variables having to do with calculating time for us to reliably be able to retrieve an accurate time-distance comparison, especially since it is so dependent on the person doing the hiking. For this reason we are temporarily relying on distance, though hopefully in a later edition of the project we will be able to take time estimates or distance.

Another major change since the proposal was using a depth-first algorithm in combination with Dijkstra instead of Held–Karp. The reason for this change was because it was too complicated and computationally inefficient to uphold the connections between trail heads. Held–Karp relies on all nodes being connected to all other nodes, which is not the case here.

The biggest constraint we took into account while doing this was practicality for the user. We went through many dry runs of what the application would need to include for it to be truly useful to a real user. A few of these constraints were: needing to end in the same spot you began, needing the spot you choose to start to be a parking lot or accessible by car (not just an edge node), and prioritizing trails that get through the most of the distance given while also making sure the user's longest path is made up of as many new nodes as possible. Trying to make this work for the user made implementing a bit more complicated than we had originally thought as the ideas motivated the way we designed for practice, not just practicality.

3 Practice Example

Run on the Silver Falls data from `testDictionary.py` starting at `Trail_5` with a 30-mile budget:

- **Data warnings print.** The script flags data-entry problems before searching, such as the `Trail_9` self-loop, the undefined `Trail_10`, one-way edges, and mismatched distances.
- **Loop report.** It prints the best round trips that start and end at `Trail_5`, ranked best first. Each line looks like `[ 9.7 mi | new nodes 9 | fresh 8.8 | repeat 0.9 ] Trail_5 -> Trail_7 -> ... -> Trail_5`.
- **History loop report (optional).** This runs only if you pass `--visited`. The same search is re-ranked so places you've already hiked don't count as new.

- **Point-to-point report.** It prints the best routes that don't have to return to the start.
- **Stats and time.** Each report ends with the number of DFS calls and any pruned branches. If a safety cap stopped the search early, it says so. A final line gives total search time, about 0.005 s on the current data.

Reading the Command Line. Take `[9.7 mi | new nodes 9 | fresh 8.8 | repeat 0.9]`:

- **mi** is the total distance walked. The line is 9.7 miles of trail.
- **new nodes** count distinct junctions visited for the first time, excluding the start (and excluding any you listed as visited).
- **fresh** is miles on trail segments not walked earlier in this route.
- **repeat** is miles on segments walked a second time. Here it's the 0.9 mi `Trail_12`↔`Trail_9` stretch walked out and back.

4 Code Work Through

The program is a depth-first search guided by Dijkstra's algorithm and a scoring rule. Because a depth-first search tries every allowed route, its work grows quickly as the trail network grows, which is why the search is pruned and capped.

1. **Build the graph** (`build_matrix_from_dict`). The trail dictionary is turned into an adjacency matrix, a grid in which the entry at row `a`, column `b` is the trail distance from point `a` to point `b`, and 0 means the two points are not directly connected.
2. **Distance home** (`dijkstra_to`). Dijkstra's algorithm, a standard shortest-path method, computes the shortest distance from every trail point back to the start. It runs once, before the search, and its results are stored in a list so the search can look up "how far is home from here?" instantly.
3. **Search** (`find_ranked_trails`). A depth-first search (DFS) starts at the origin and follows one trail as far as the rules allow, then backs up (backtracks) and tries the next option, until every allowed route has been explored. It follows these rules:
   - A route may not immediately turn around, except at a true dead end (a point with only one trail), such as a spur trail.
   - A directed trail segment can be used only once.
   - A junction can be visited at most twice.
   - It abandons (prunes) any step where the distance walked, plus the step, plus the shortest way home, would exceed the budget. This is where Dijkstra pays off. No route that could still finish within the budget is ever discarded by this check, because no way back is shorter than the shortest way home.
4. **Record.** When a loop returns to the start (or at any point, for point-to-point routes), the route is scored and added to a list of the best 25 routes. That list is kept as a heap, a structure that makes it cheap to find and drop the current worst entry when a better route arrives. A route and its exact mirror image count as one.
5. **Score** (`score_route`). It computes the new nodes, fresh miles, and repeat miles described above. Routes are compared on new nodes first, then fresh miles, then fewer repeat miles.

6. **Prune and cap.** Once the best-25 list is full, the branch-and-bound step estimates the most new nodes a branch could still reach, counting every unvisited point that is reachable with the remaining distance. If that best case is lower than the 25th-best route's count, the branch is skipped. Because the estimate is optimistic, skipping a branch never discards a route that could have made the list. The call and time caps guarantee the search stops.
7. **Results** (`show`). It formats the results according to their score.

5 Verification

We tested the search in two ways. First, we compared it with a brute-force check, a separate and simpler program that applies the same trail rules but has no Dijkstra pruning, no scoring, and no top-25 list, so it simply lists every loop that fits within the budget. Table 1 shows that, on the Silver Falls data starting at `Trail_5`, the two found exactly the same set of loops at every budget we tried. Second, we checked every loop the search returned: each one starts and ends at `Trail_5`, stays within the budget, reuses no directed trail segment, and has a reported distance equal to the sum of its segments. We also confirmed that branch-and-bound pruning returns the same ranking as the search without it. On this small network the pruning never actually triggers (the 30-mile run pruned no branches), so that last check will only become meaningful on larger data.

Budget (mi)	Brute-force loops	Algorithm loops	Identical sets
6	7	7	yes
8	13	13	yes
10	47	47	yes
12	71	71	yes
30	77	77	yes

Table 1: Loop counts from the brute-force check versus the algorithm.

Because the brute-force check uses the same trail rules, it shows that the search, scoring, and mirror-image de-duplication work as intended, but not that the rules themselves match real hiking behavior.

6 What Remains

We need to implement a much larger data set, and add the complete map of trails in Silver Falls: only one map implemented so far. We also need to do some of the math for time-distance comparisons. There is also no user prompting yet, so that system of questions will need to be available in our final draft of the project.

7 Limitations

Our runtime understanding based on our tests is in a non-perfect environment so it is likely that some data was skewed in our preliminary results due to other things running in the background. Another limitation is that the current draft of the project is only tested on a small set of nodes, so we may need to do some more rearranging to optimize our runtime on larger data sets.

8 Contributions

Grace: Contributed to: initial research, project proposal, progress report editing, implementation of Dijkstra and Held–Karp and orienteering problem, creating the initial `depthFirst_prototype.py`, formatting the README

Lucy: Contributed to: initial research, project proposal, progress report, implementation of Dijkstra, planning phase (research of Held–Karp and orienteering problems, pruning and optimizing `depthFirst_prototype.py`, BST and linked list possibilities)

Hannah: Contributed to: initial draft of proposal, initial data analysis, initial planning, collecting data (state park trails)